

THE LAYER IS NOT ATOMIC: PARTITIONING THE RESIDUAL STREAM FOR COMPUTE-EFFICIENT TRANSFORMERS

Anonymous authors

Paper under double-blind review

ABSTRACT

The transformer layer is monolithic by convention: a single attention block and a single feed-forward network, both of width D , process every token at every layer. A growing line of work decomposes this layer into several narrower parallel blocks, but every such design pays for the narrowing with learned down- and up-projections into and out of the reduced width, and buys back the cost with routing sparsity. We show the payment is avoidable. In the *Modular Sub-Transformer* (MST), each layer contains N parallel sub-transformers of width $d = D/N$ that *partition* the residual stream: the N sub-states, concatenated, *are* the D -dimensional representation, so no projection is required to enter or leave a narrow block. Sub-transformers exchange information once per layer through a routed d -dimensional bottleneck with a relu^2 expansion. Because the per-sub head budget satisfies $n_h \cdot d_h = d$, attention FLOPs and KV cache size are preserved *exactly*, while per-layer matmul parameters fall by $3.20\times$ at $N=4$. The model remains fully dense: every sub-transformer runs for every token, so there is no expert dispatch, no capacity constraint and no load-balancing sparsity to tune. Against a strongly-tuned dense baseline (Muon, μP , QK-norm, value embeddings, sliding-window attention) on a compute-optimal token ladder, MST is Pareto-dominant on three axes simultaneously over $199\text{M} \rightarrow 914\text{M}$ parameters: matching MST’s bits-per-byte requires $1.77\times$ the parameters, $1.35\times$ the inference FLOPs per token, or $2.00\times$ the training compute. **[TODO: add one downstream + one wall-clock number to the abstract once Tables 4 and 5 are filled]**

1 INTRODUCTION

Scaling a transformer language model means growing two numbers: depth L and width D . The layer itself is treated as an indivisible unit: one attention block and one FFN, both spanning the full D channels. Every token is processed by all D^2 -scale weight matrices at every layer, and no structural distinction is drawn between different parts of the representation.

Recent work has begun to question this. Mixture-of-Experts (Shazeer et al., 2017; Fedus et al., 2022) routes tokens to alternative FFNs, but each expert still reads the full D -dimensional token and the layer remains full-width. Layer-level modularity, as in Mixture-of-Depths (Raposo et al., 2024), MoEUT (Csordás et al., 2024) and Mixture-of-Recursions (Bae et al., 2025), changes *which* full-width blocks execute. Most directly, Mixture-of-Layers (MoL) (Ternovtsii & Bilak, 2026b) replaces full-width blocks with K parallel *thin* blocks of width $d_{\text{thin}} \ll D$, composed by top- k routing.

MoL isolates the right question and, in our reading, also exposes the cost of the standard answer. Because its thin blocks must read from and write to a full-width residual stream, each is wrapped in learned projections $W_{\text{down}} \in \mathbb{R}^{d_{\text{thin}} \times D}$ and $W_{\text{up}} \in \mathbb{R}^{D \times d_{\text{thin}}}$. That wrapper consumes 40% of block parameters at $d_{\text{thin}}=256$ and 73% at $d_{\text{thin}}=64$. This is a *projection tax* that grows precisely as the blocks get narrow enough to be interesting. The tax is paid back with sparsity: only k of K blocks run per token, which recovers active-compute efficiency but forfeits total-parameter efficiency. At 1.3B scale MoL reports a 3.01 PPL deficit against an iso-total-parameter dense baseline, and a 1.9–2.3 $\times$ training-time premium.

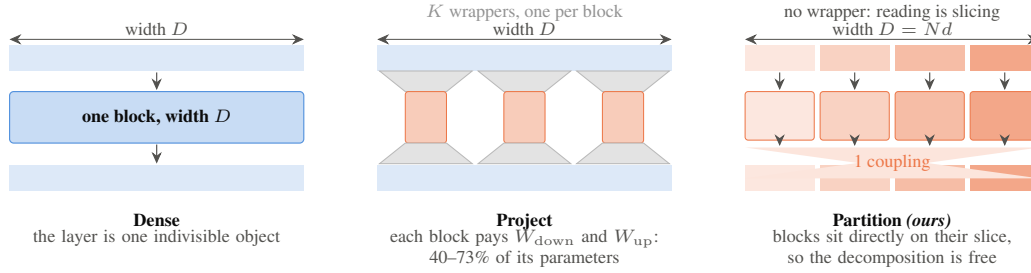


Figure 1: Three ways to make a transformer block narrow, all drawn at the same total width D . Projecting into a narrow block costs a $W_{\text{down}}/W_{\text{up}}$ wrapper on *every* block. Partitioning costs nothing, because each block sits directly on its own slice of the stream and the N slices concatenated are the representation. The single coupling that replaces the wrappers is detailed in Figure 2.

We ask whether the projection can be removed rather than amortized. Our answer is a change of viewpoint: instead of projecting a full-width residual stream *into* narrow blocks, let the narrow blocks *be* the residual stream. If a layer holds N sub-transformers of width $d = D/N$, and sub-transformer i owns channels $[id, (i+1)d)$ of the representation, then reading and writing are free: they are slicing and concatenation. The decomposition costs nothing, and what remains to be designed is only how the sub-transformers communicate.

Our contributions:

1. **A projection-free layer decomposition.** MST partitions the residual stream into N sub-streams processed by independent width- d transformers, coupled once per layer by a routed bottleneck (Section 3). Per-layer matmul parameters fall from $12D^2$ to $3.75D^2$ at $N=4$, while attention FLOPs and KV cache are preserved *exactly by construction*, not approximately.
2. **Pareto dominance without sparsity.** On a compute-optimal ladder from 199M to 914M parameters, MST dominates a strong dense baseline on total parameters, inference FLOPs per token, and total training compute simultaneously (Section 5). Unlike MoE and MoL, MST is dense: all sub-transformers are active for all tokens, so the parameter win is not traded against active compute.
3. **The optimization of partitioned weights matters as much as the architecture.** A naively-trained partition suffers a $7.2\times$ imbalance in per-sub gradient norm and cross-sub contamination in the orthogonalizing optimizer. Two fixes, per-sub gradient-norm equalization and block-diagonal Newton–Schulz, together with a widened transition bottleneck, recover $\sim$ BPB at $D=512$ (Section 6).
4. **Coupling is necessary; its topology is not.** Removing the learned transition costs 0.039 BPB, yet eleven distinct attempts to enrich it (nonlinear, gated, MLP, bilinear, per-slice routed, multi-layer lookback, cross-sub gating, cross-sub KV injection, low-rank query modulation, feature cycling, and a global residual stream) all fail to improve on it (Section 6). This is an independent dense-architecture analogue of the routing-equifinality result of Ternovtsii & Bilak (2026a).

2 RELATED WORK

Narrow parallel blocks. MoL (Ternovtsii & Bilak, 2026b) is the closest work and the sharpest contrast. It replaces full-width blocks with K thin transformer blocks at reduced d_{thin} , selected per token by top- k routing, and adds a shared full-attention block because sparse dispatch leaves each routed block seeing only a fraction of the sequence. The design is motivated by the same observation as ours, that the layer need not be monolithic, but it answers differently on three counts. Its thin blocks read from and write to a full-width residual stream, so each carries a $W_{\text{down}}/W_{\text{up}}$ wrapper costing 40–73% of the block; it recovers that cost through sparsity, activating k of K blocks; and it substitutes linear attention in the routed blocks. The consequences are visible in its own results: at

1.3B, MoL leads an iso-active dense baseline by 0.49 PPL but trails an iso-*total* one by 3.01 PPL, and trains 1.9–2.3 $\times$ slower. MST removes the wrapper instead of amortizing it, stays dense, and keeps softmax attention throughout, which is why it can be compared on total parameters at all. UoE (Yang et al., 2025) decomposes attention and MLP into expert groups with hierarchical routing, and shares MoL’s reliance on a full-width stream and on sparsity.

Residual-stream structure. Hyper-Connections (Zhu et al., 2025; Xie et al., 2026) expand the residual into n full-width streams to improve optimization, at roughly $n \times$ residual memory; AltUp (Baykal et al., 2023) widens the token representation and updates one block per layer; Multi-Stream Transformers (Burtsev & Rumshisky, 2021) split an encoder into parallel streams merged at the end. All of these *widen*. MST *narrows* N streams to D/N and spends the saving on depth. Concurrent work on FFN shape (Anonymous, 2026b) and channelized architectures (Anonymous, 2026a) restructures within a layer but leaves the residual width convention intact.

Partial partitions. MH-MoE (Wu et al., 2024) splits tokens into sub-tokens routed to different experts and FlashMHF (Zhang et al., 2026) builds parallel narrow sub-networks inside the FFN; both are FFN-only and re-merge within the layer. Layer-level modularity (Raposo et al., 2024; Csordás et al., 2024; Bae et al., 2025) changes which full-width blocks execute rather than their width. MST partitions attention as well, and the partition persists across the full depth.

Structured matrices and attention spans. An MST layer is by construction a block-diagonal weight matrix with N blocks plus a low-rank cross-block correction, placing it in the family of Monarch (Dao et al., 2022) and BLAST (Lee et al., 2024), but applied to a whole layer rather than to individual linear maps and with a learned, input-dependent correction. Assigning different context windows to different heads or layers is likewise established (Sukhbaatar et al., 2019; Beltagy et al., 2020; Fu et al., 2024; Zhang et al., 2025); MST lifts it from the head to the stream, so a window governs a block’s attention, its FFN and its slice of the residual jointly.

3 METHOD

Notation. D is the model width, N the number of sub-transformers, $d = D/N$ the sub width, and L the depth. The state at layer ℓ is $H^{(\ell)} \in \mathbb{R}^{B \times T \times N \times d}$, its flattening over the last two axes is the ordinary D -dimensional residual stream. We write $X_i \in \mathbb{R}^d$ for the i -th sub-state at a given token and suppress the batch and time axes throughout. Each sub-transformer uses n_h heads of dimension d_h with $n_h d_h = d$.

3.1 THE LAYER

Per stream: nothing new. Sub-transformer i owns X_i and no other channel. It runs an ordinary pre-norm transformer block at width d , with its own attention window w_i , RoPE (Su et al., 2021) and QK-norm:

$$X_i \leftarrow X_i + W_i^O \text{Attn}_{w_i}(W_i^Q \tilde{X}_i, W_i^K \tilde{X}_i, W_i^V \tilde{X}_i), \quad X_i \leftarrow X_i + W_i^{F2} \text{relu}^2(W_i^{F1} \tilde{X}_i), \quad (1)$$

where $\tilde{X}_i = \text{RMSNorm}(X_i)$ is recomputed before each sublayer. This is deliberately unremarkable: it is the dense baseline’s block with D replaced by d everywhere. Because no term reads a channel that sub i does not own, the N blocks together form a block-diagonal factorization of one D -width layer rather than N copies of a smaller model.

Coupling: the whole mechanism, in one line. The streams exchange information exactly once per layer. Everything they say to each other passes through a single d -dimensional vector:

$$X_i \leftarrow X_i + \underbrace{\overbrace{W_i^D}^{\text{scatter } 1 \rightarrow N} W^\downarrow \text{relu}^2 \left(W^\uparrow \underbrace{\sum_j p_j \tilde{X}_j}_{\text{aggregate } N \rightarrow 1} \right)}_{\text{bottleneck } d \rightarrow D \rightarrow d}, \quad p = \text{softmax} \left(W^R \frac{1}{N} \sum_j \tilde{X}_j \right). \quad (2)$$

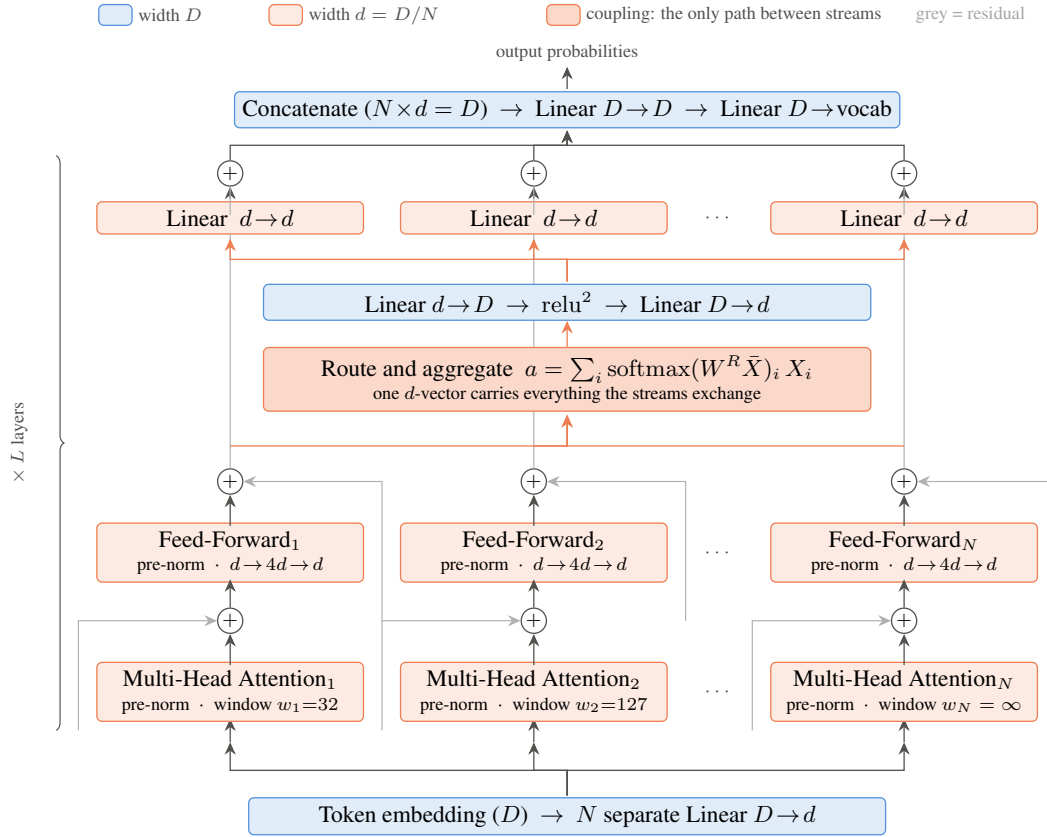


Figure 2: One MST layer, read bottom to top. Colour encodes width and nothing else: blue operates at D , orange at $d = D/N$. The N streams run ordinary transformer sublayers in parallel and never read each other’s channels; the coupling is the only path between them, and everything they exchange passes through the single d -vector a . Grey lines are residual connections, which carry each stream past the coupling unchanged.

Read right to left, Eq. 2 is the hourglass of Figure 2: N streams collapse to one d -vector, that vector is widened to D for a single nonlinearity and squeezed back to d , and N distinct read-out matrices deliver it. Parameter shapes and counts are in Appendix A.

Three consequences follow directly from the form of Eq. 2.

(i) *The router selects nothing.* p lies in the simplex over *sub-transformers*, not over tokens-to-experts. Every stream runs for every token, so MST has no sparse dispatch, no capacity factor, and no dropped tokens. A light load-balancing term $\alpha N \sum_i \bar{p}_i^2$ with $\alpha=0.01$ (Fedus et al., 2022) is minimized at uniform p .

(ii) *The N read-outs are what break the symmetry.* Every stream receives the same vector. Through a shared W^D they would receive identical messages and could diverge only through initialization noise. The distinct W_i^D make the message stream-specific, and they are the cheapest term in the layer (Appendix A).

(iii) *The layer is exactly the identity at step 0.* W_i^O , $W_i^{F^2}$ and $W^\downarrow$ are zero-initialized and W^R starts at zero, so every residual branch contributes nothing and p is exactly uniform. Training begins from a well-conditioned deep identity, not from a random partitioned map.

The head budget is conserved. Each sub spends $n_h d_h = d$ on attention, so the layer spends $Nd = D$, exactly as a dense layer does. This one identity is why attention FLOPs and KV cache are

preserved while only the matmul parameters shrink (Section 3.4). Our dense baseline uses $d_h=128$; MST uses $d_h=128/N=32$ with N times as many heads.

3.2 MULTI-SCALE SUB-WINDOWS

Sub-transformer i attends with a causal sliding window w_i spaced geometrically from $w_{\min}=32$ to full context, the last sub always unrestricted. For $N=4$ at $T=2048$ this gives $w = (32, 127, 511, \infty)$. This is the only component of MST that supplies an *a priori* reason for sub-transformers to differ; every other source of differentiation is symmetry breaking through W_i^D and initialization.

The assignment is close to FLOP-neutral, so it is a prior rather than a shortcut. Summed effective context is $32 + 127 + 511 + 2048 = 2718$ per layer against $4 \times 704 = 2816$ for the dense baseline’s SSSL layer-alternating pattern, a 3.5% reduction.

3.3 OPTIMIZING PARTITIONED WEIGHTS

The N matrices of each type are stored fused as one $(N \cdot \text{out}) \times \text{in}$ tensor and applied with a batched matrix multiply. That is an implementation detail for throughput, but it exposes two training problems that a dense model does not have.

Gradient imbalance. Measured on the $N=4$ model, per-sub gradient norms differ by $7.2\times$: sub 0 receives 55% of the total and sub 3 receives 7.6%. The mechanism is a feedback loop through the concatenating output head, which weights one sub more, which trains it faster, which weights it more still. We equalize with a backward hook that views the fused gradient as $(N, \text{out}, \text{in})$ and rescales each sub to the mean norm. Only magnitudes are touched; directions are left alone.

Cross-sub contamination in Muon. Muon orthogonalizes the update via a Newton–Schulz / Polar-Express iteration (Jordan et al., 2024; Amsel et al., 2025). Applied to the fused tensor it mixes gradient directions belonging to different sub-transformers, which is exactly what the partition is designed to keep apart. We run the iteration independently on each of the N blocks. Because Muon scales its step by $\max(1, \text{rows}/\text{cols})^{1/2}$, blocking also divides the effective learning rate on the fused attention tensors by $\sqrt{N}$; we restore it with a per-sub multiplier of $\sqrt{N}$, which simultaneously corrects the μP (Yang et al., 2021) mismatch between a D -tuned rate and d -wide matrices. The two interventions are therefore coupled and we ablate them jointly (Section 6).

3.4 COST ACCOUNTING

Parameters. Summing the components of Eq. 2 and the per-stream sublayers (Appendix A gives the term-by-term budget),

$$\underbrace{\frac{12D^2}{N}}_{N \text{ sub-transformers}} + \underbrace{\frac{3D^2}{N}}_{\text{coupling}} = \frac{15D^2}{N} \text{ per layer, against } 12D^2 \text{ dense,} \quad (3)$$

a reduction of $0.8N$ rather than N : the coupling consumes a quarter of what the partition saves. It is nonetheless far cheaper than a projection wrapper, which costs $2Dd$ *per block* rather than once per layer.

Attention FLOPs and KV cache. A sub contributes $12 n_h d_h T_{\text{eff}} = 12 d T_{\text{eff}}$ per token per layer; summed over N subs this is $12 D T_{\text{eff}}$, the dense value exactly. The KV cache is $Nd = D$ per layer, again exactly dense. MST therefore takes its saving entirely from matmul parameters and inherits neither an attention advantage nor an attention penalty, which is what makes the FLOPs-per-token axis of Section 5 a clean comparison.

Value embeddings. Per-layer value embeddings (Zhou et al., 2024) in MST live at width d and are shared across subs with per-sub gates, so they are $N\times$ smaller than the dense baseline’s. Since the dense baseline spends 44% of its total parameters on value embeddings at $L=24$, we report transformer-matrix parameters alongside total parameters throughout (Table 1) so that this effect can be separated from the layer decomposition itself.

Table 1: Cost and quality. MST is $N=4$ with all components of Section 3. Dense is the matched baseline at the same $D = 64L$ ladder. “Matrices” excludes embeddings and value embeddings.

Model	L	Total params	Matrices	FLOPs/token	Train FLOPs	BPB↓
MST	8	58,726,416	8.4M	1.844e8	4.875e16	1.0510 [‡]
	9	82,809,618	14.7M	2.607e8	9.752e16	1.0000 [‡]
	16	199,254,048	65.0M	7.251e8	7.507e17	0.8810
	18	252,696,996	92.3M	9.492e8	1.296e18	0.8544
	20	314,938,920	126.2M	1.218e9	2.150e18	0.8349
	22	386,717,100	167.6M	1.535e9	3.444e18	0.8104
	24	468,768,816	217.1M	1.905e9	5.350e18	0.7946
	26	561,831,348	275.6M	2.333e9	8.087e18	0.7804
Dense	32	914,456,640	511.8M	4.008e9	2.436e19	0.7434
	8	125,829,648	25.2M	2.863e8	1.261e17	0.9691
	12	286,262,424	84.9M	7.385e8	8.537e17	0.9030
	14	399,115,836	134.9M	1.101e9	1.899e18	0.8688
	16	536,872,992	201.3M	1.548e9	3.817e18	0.8364
	18	701,893,188	286.7M	2.134e9	7.269e18	0.8131
	20	896,535,720	393.2M	2.827e9	1.292e19	0.7906
	22	1,123,159,884	523.4M	3.694e9	2.209e19	0.7714
	24	1,384,124,976	679.5M	4.690e9	3.594e19	0.7545
	26	1,681,790,292	863.9M	5.894e9	5.684e19	0.7382
	28	2,018,515,128	1079.0M	7.250e9	8.661e19	0.7246
	30	2,396,658,780	1327.1M	8.847e9	1.291e20	0.7120

[‡] below the amortization threshold; excluded from the fits of Table 3 and reported as a sensitivity in Section 8. All cost columns are recomputed from a single implementation at one commit, so the two arms are directly comparable; the token budget is 10.5 tokens per scaling parameter and Train FLOPs is FLOPs/token $\times$ tokens.

4 EXPERIMENTAL SETUP

Baseline. Our dense baseline is not a vanilla transformer. It is a speedrun-tuned decoder (Karpathy, 2025) with RMSNorm pre-norm, RoPE, QK-norm, relu^2 FFN, per-layer value embeddings, layer-alternating sliding-window attention, logit soft-capping, learned per-layer residual scalars, Muon for matrices with AdamW for embeddings and scalars, μ P learning-rate transfer, and a batch-size schedule following Bergsma et al. (2025). MST changes only the layer structure; every other component, hyper-parameter and schedule is shared.

Ladder. $D = 64L$ rounded up to a multiple of 128; $N=4$; $T=2048$; vocabulary 32,768. Both arms are trained at 10.5 tokens per scaling parameter (transformer matrices plus output head), so each model receives its own compute-optimal budget (Hoffmann et al., 2022); MST consequently trains on fewer tokens than dense at equal depth. Data is a filtered web corpus (Penedo et al., 2024), single epoch.

Metric. Bits per byte on held-out data, which is invariant to tokenizer vocabulary.

5 RESULTS

5.1 PARETO DOMINANCE

Figure 3 shows MST below the dense curve on all three axes over the entire measured range. Table 2 quantifies the gap by inverting the dense fit: for each MST model we report the resource a dense model would require to reach the same BPB.

5.2 SCALING EXPONENTS

Fitting $\text{BPB} = ax^b$ (Table 3), MST’s exponents are marginally steeper on all three axes, but the difference is within fit uncertainty for seven points and we do *not* claim a scaling-exponent advantage.

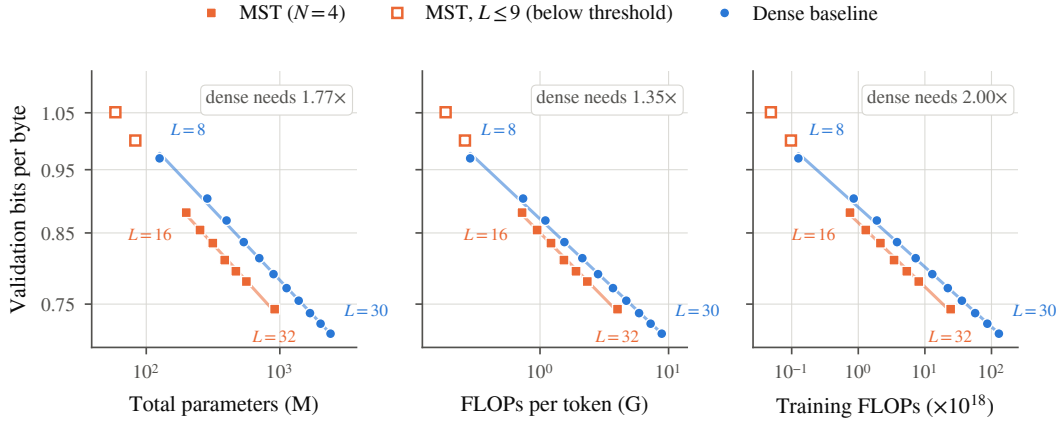


Figure 3: MST lies below the dense scaling curve on all three cost axes. Points are measured runs; lines are power-law fits (Table 3). Hollow markers are MST models below the amortization threshold, which are excluded from the MST fit. Each panel states the multiplier a dense model needs to reach the same bits-per-byte, averaged over $L \in [16, 32]$.

Table 2: Resource multiplier a dense model needs to match each MST model, obtained by inverting the dense power-law fit. The multiplier is stable across the range.

MST L	16	18	20	22	24	26	32	mean
Total parameters	1.66	1.74	1.73	1.86	1.84	1.82	1.75	1.77×
FLOPs / token	1.22	1.30	1.30	1.42	1.42	1.41	1.39	1.35×
Training FLOPs	1.64	1.86	1.85	2.22	2.20	2.16	2.08	2.00×

The defensible statement is that the multiplier of Table 2 is approximately constant over a $4.6\times$ span of parameters and a $32\times$ span of training compute. Including models below 100M ($L \leq 9$) would widen the exponent gap substantially, but those points lie *above* the dense curve on the compute axes and steepen the fit artificially; we report that crossover in Section 8 rather than fitting through it.

5.3 DOWNSTREAM TRANSFER

Perplexity gaps of this size need not translate into downstream ability, so we evaluate zero-shot transfer against two dense reference points that bracket MST: one matched on total parameters and one matched on FLOPs per token (Table 4). --

5.4 THROUGHPUT, MEMORY AND HARDWARE UTILIZATION

FLOP counts are not wall-clock. MST trains $1.20\times$ faster than dense while reaching less than half its utilization (Table 5): it wins on throughput by issuing $2.46\times$ fewer FLOPs, not by using the machine well.

The utilization gap has two distinct causes. Measuring fused attention in isolation, with the total attention width held constant and no MST code in the loop, gives 171.9, 283.9 and 365.7 TFLOP/s at $d_h = 32, 64$ and 128 (Figure 4). A partition into N streams forces $d_h = 128/N$, so at $N=4$ a $2.13\times$ penalty on the attention portion is intrinsic to the architecture given current kernels. The remainder is ours: the layer issues N separate attention calls, one per stream, where a single call over Nn_h heads would compute the same thing whenever the streams share a window, since attention is independent across heads. At $L=24$ that is 96 launches per forward pass where 24 would do. We report the measured kernel penalty and do not claim to have eliminated the implementation cost.

Table 3: Power-law fits $\text{BPB} = ax^b$, MST $L \in [16, 32]$ and dense $L \in [12, 30]$.

Axis x	Dense a, b	R^2	MST a, b	R^2
Total parameters	$7.291 x^{-0.1077}$	0.998	$7.446 x^{-0.1119}$	0.996
FLOPs / token	$5.904 x^{-0.0923}$	0.998	$6.713 x^{-0.0997}$	0.996
Training FLOPs	$5.905 x^{-0.0457}$	0.998	$6.590 x^{-0.0490}$	0.996

Table 4: Zero-shot accuracy (%). MST is compared against a dense model matched on total parameters and a dense model matched on FLOPs per token.

Task	MST $L=32$ (914M)	Dense $L=20$ (iso-param)	Dense $L=22$ (iso-FLOP)
ARC-Easy (10-shot)	66.8	--	--
PIQA (10-shot)	73.3	--	--
HellaSwag (10-shot)	51.6	--	--
Winograd (0-shot)	63.4	--	--
LAMBADA (0-shot)	40.0	--	--
OpenBook QA (0-shot)	40.0	--	--
ARC-Challenge (10-shot)	37.5	--	--
CORE metric	0.231	--	--

Seven of the 22 CORE tasks, chosen as those with meaningful headroom at this scale; the CORE metric aggregates all 22. Tasks on which a model of this size sits at chance (CommonsenseQA, WinoGrande, Jeopardy, BigBench repeat-copy) carry no signal here and are deferred to Appendix D. **[TODO: dense columns pending: $L=20$ is $0.98\times$ MST total params, $L=22$ is $0.92\times$ MST FLOPs/token]**

Memory. MST’s peak memory is 8% *higher* than dense despite $3.2\times$ fewer per-layer parameters. The cause is the batched formulation rather than the partition: applying N stacked weight matrices requires a permutation before the batched matrix multiply, which forces a contiguous copy of a full activation tensor, and this happens for each of the six batched projections per layer. Parameter count shrinks with N ; activation traffic does not. The same extra data movement lowers arithmetic intensity, so the memory overhead and the utilization gap share a cause rather than one explaining the other.

Figure 4 attributes the utilization gap to the head-dimension regime rather than to the architecture, and therefore identifies it as implementation headroom.

6 ABLATIONS

Ablations are run at $L=8$ ($D=512$), where a full grid is affordable, with three seeds per condition. A $L=16$ grid would cost roughly $40\times$ as much and we did not run one.

Ablation transfer. Small-scale ablations are only useful if their conclusions survive at the scales the paper actually claims. We can test this directly for the one intervention we measured at two depths: the full training recipe of Section 3.3 applied to the plain partition. It preserves its sign and, to within 15%, its magnitude (Table 6). We take this as evidence that the $L=8$ ordering is informative about $L=16$, while noting that one intervention at two depths is a weak test and that we cannot rule out an ablation whose sign flips with scale.

Granularity. $N=4$ substantially outperforms $N=8$ at equal D (--- versus --- BPB). This matches MoL’s independent finding that block *width* dominates block *count* at fixed parameters, and runs opposite to the fine-grained-expert trend in FFN-MoE (Dai et al., 2024). Both observations are consistent with the interpretation that when a block contains attention as well as an FFN, per-block capacity matters more than combinatorial flexibility.

Table 5: Measured throughput and memory. Weights are randomly initialized; these are architecture-level measurements and do not depend on training.

Measurement ($L=24$, H200)	MST	Dense	
Training step, fwd+bwd (ms)	420.4	504.4	MST $1.20\times$ faster
Achieved throughput (TFLOP/s)	297	609	
Model FLOPs utilization (%)	30.0	61.6	dense $2.05\times$ higher
Peak memory (GB)	115.3	106.7	MST 8% higher
KV cache per layer	D	D	identical by construction
Decode latency (ms/token)	--	--	[TODO: pending]

Batch 32, $T=2048$, `torch.compile` enabled for both arms as in training, minimum over 20 iterations. Weights are randomly initialised: these are architecture-level measurements independent of the trained values.

[TODO: Fig. 3: (a) fused attention throughput vs d_h at constant total width: 171.9 / 283.9 / 365.7 TFLOP/s for $d_h = 32/64/128$; (b) MST kernel-time breakdown.]

Figure 4: The utilization gap decomposes into a kernel effect and an implementation cost. Fused attention at $d_h=32$, which $N=4$ forces, reaches only $2.13\times$ less throughput than at $d_h=128$ with the total attention width held fixed and no MST code involved.

Enriching the coupling does not help. Beyond the routed bottleneck we tried a nonlinear (SiLU) bottleneck, an input-gated router, a concat-MLP-split transition, a bilinear second-order aggregate, per-slice independent routing, DenseFormer-style multi-layer lookback, EMA hyper-connected sub-residuals, cross-sub FFN gating, per-token cross-sub KV injection, low-rank cross-sub query modulation, cyclic feature permutation across subs, and a D -wide global residual stream. None improved on the routed bottleneck (Appendix B reports all twelve). Removing the coupling entirely, however, costs 0.039 BPB. Coupling is necessary; its form appears not to be. This mirrors Ternovtsii & Bilak (2026a), who reach an equifinality conclusion for routing topology in sparse MoE, and extends it to a dense partitioned setting.

7 ANALYSIS

The $7.2\times$ gradient imbalance of Section 3.3 raises the concern that a partitioned model silently degenerates into a narrower one. Two inference-time probes on the $L=32$ model say otherwise (Figure 5).

Every stream is load-bearing. Zeroing any single stream at every layer raises bits-per-byte from 0.710 to between 5.15 and 8.48: the model is destroyed whichever stream is removed. The intervention is deliberately blunt, since a quarter of the residual stream is deleted at every layer, so the magnitude is uninformative and the *spread* is the result. Damage varies by only $1.75\times$ across the four streams, against the $7.2\times$ gradient imbalance the recipe was built to correct. No stream is close to redundant.

Streams divide along token frequency. Attributing the damage of the previous probe token by token, and normalizing each stream by its own mean so that overall importance is factored out, gives a monotonic split across frequency quartiles (Figure 5b). Relative to its own average, stream 2 matters $1.78\times$ more than usual on the rarest quartile and $0.30\times$ on the commonest, a $5.9\times$ swing; stream 1 runs the other way, $0.58\times$ to $1.40\times$. All four profiles are monotonic. An independent probe agrees: decoding each stream alone through the output head, stream 1 attains 3.52% top-1 agreement with the full model against 0.03–0.24% for the others (Figure 5c), and the tokens it promotes are

Table 6: Ablation transfer. The one intervention measured at two depths keeps its sign and approximate magnitude, which is the basis on which we run the remaining ablations at $L=8$ only.

Condition	$L=8$ (BPB)	$L=16$ (BPB)
Plain partition (no recipe)	1.0798	0.9057
+ full recipe (Section 3.3)	1.0510	0.8810
Δ	-0.0288	-0.0247

same sign, $0.86\times$ magnitude

Table 7: Ablations at $L=8$, $D=512$, $N=4$, mean $\pm$ s.d. over 3 seeds. Negative Δ is better.

	Condition	BPB	Δ
Controls	Dense baseline ($d_h=128$)	0.9592 ± 0.0002	n/a
	Dense baseline, $d_h=32$ (<i>head-dim control</i>)	0.9760 ± 0.0003	+0.0168
Coupling	No coupling (independent columns)	--	--
	Parameter-free mean coupling	--	--
	Routed bottleneck (ours)	--	n/a
Recipe	Plain partition (no recipe)	--	--
	+ gradient equalization	--	--
	+ block-diagonal Muon & per-sub LR	--	--
	+ wide transition	--	--
Full	All of the above	--	--
	+ multi-scale sub-windows (MST)	--	--
Granularity	$N=2$	--	--
	$N=4$	--	n/a
	$N=8$	--	--

[TODO: Tier-3 grid; the head-dim control is the load-bearing row] Block-diagonal Muon and the per-sub learning rate are ablated as a unit because blocking divides the effective rate by $\sqrt{N}$ and the multiplier restores it; the two separately, and the bottleneck-width sweep, are in Appendix C.

dominated by punctuation and line breaks, which is the highest-frequency class. Two probes with no mechanism in common therefore place the same stream at the same end of the same axis.

We report this as evidence that the partition does something structured rather than as an interpretability claim, and one caveat limits how far it can be pushed: deleting a stream raises token loss by roughly 20 nats, well above the $\log V = 10.4$ nats of a uniform predictor, so the perturbed model is confidently wrong rather than merely uninformed. The comparison is between catastrophic failures, and the monotonicity is what carries the signal. A graded perturbation would support a stronger claim.

In weight space the streams are genuinely distinct. The activation statistic above compares vectors living in different channel ranges, which limits what it licenses. The N distribute matrices do not have that problem: they all map from the *same* aggregate vector, so cosine between them is well posed. Mean pairwise cosine among the W_i^D is +0.040 overall and decays from +0.079 at the first layer to +0.015 at the last, while the per-stream W^Q , W^K , W^V and W^{F1} sit at $|\cos| < 0.0015$, indistinguishable from the ≈ 0.002 expected of unrelated matrices of this size (Figure 5f). The symmetry breaking of Section 3 works, and the streams diverge further with depth.

Streams stay near-orthogonal, and the coupling summarizes rather than selects. Mean pairwise cosine similarity between streams is close to zero through the body of the network, between -0.09 and +0.08 for layers 8–19, rising to +0.64 at layer 2 and to +0.16 near the output. It does not approach the $-1/(N-1) = -0.333$ floor that maximal mutual repulsion would produce: the streams are approximately independent rather than actively anti-correlated, and they grow more alike at both ends of the network, where they sit closest to the shared embedding and the shared output

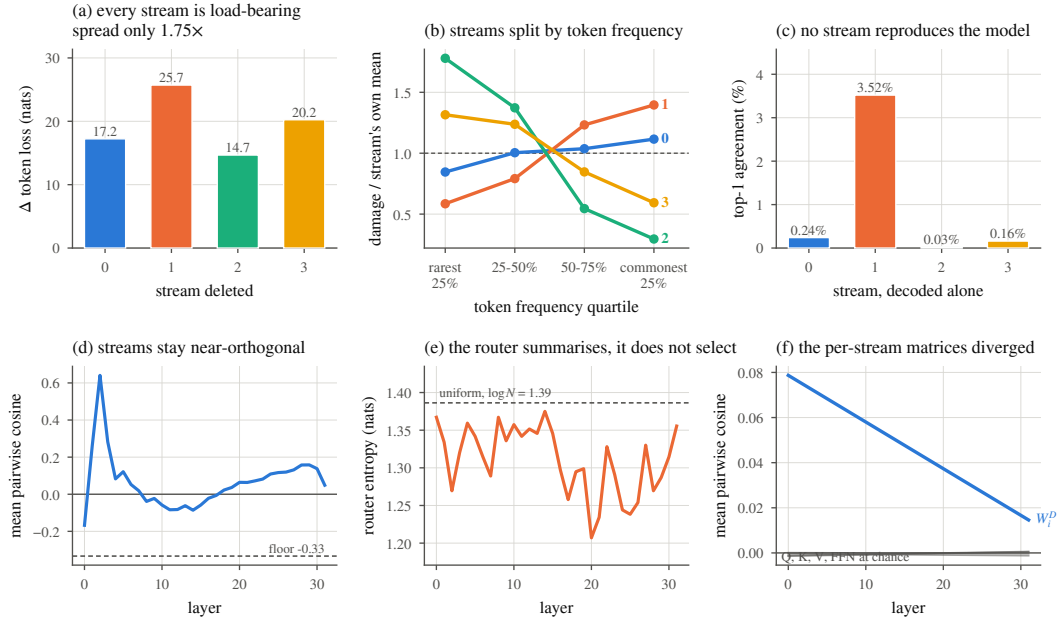


Figure 5: Stream-level analysis of the $L=32$ model. **(a)** deleting any stream destroys the model, and the spread across streams is only $1.75\times$. **(b)** normalising each stream by its own mean reveals a monotonic split along token frequency: streams 2 and 3 carry rare tokens, streams 0 and 1 carry common ones. **(c)** decoded alone, no stream reproduces more than 3.5% of the model’s predictions. **(d,e)** activation similarity and router entropy per layer, against the $-1/(N-1)$ floor and $\log N$. **(f)** in weight space the per-stream attention and FFN matrices are uncorrelated at every depth, while the distribute matrices share a small component that decays with depth.

head. Router entropy stays between 1.21 and 1.38 against a maximum of $\log N = 1.386$, within 13% of uniform at every layer, so the coupling acts as a summarizer rather than a selector.

Two caveats on these diagnostics. First, near-uniform routing is partly *enforced*: the load-balancing term of Section 3 penalizes $\sum_i \bar{p}_i^2$ and is minimized at the uniform p , so high entropy is evidence that the regularizer works, not that a free router would choose to average. What the entropy does establish is that whatever the router contributes (the parameter-free mean ablation of Section 6 puts that at 0.039 BPB) it contributes through small deviations from uniform rather than through selection. Second, the similarity statistic compares stream i ’s mean direction with stream j ’s, but the two vectors occupy different channel ranges of the model and there is no reason for index k to denote the same feature in both. The measure is therefore asymmetric in what it licenses: a high value is strong evidence of redundancy, whereas a value near zero is only consistent with independence and does not establish it, since two streams could show zero mean-direction similarity while being strongly correlated token by token. The informative datum here is the $+0.64$ at layer 2, not the near-zero plateau.

Device parallelism. Streams have no within-layer cross-stream data dependency, so the N columns of a layer can be placed on different devices with a single d -wide all-reduce at the coupling. That exchange is narrower than the two D -wide all-reduces per layer of tensor parallelism (Narayanan et al., 2021), and it avoids the all-to-all token dispatch that expert parallelism requires. **[TODO: optional: measured 2-GPU numbers]**

8 LIMITATIONS

Range of validity. The efficiency multipliers of Table 2 are measured from 199M to 914M total parameters and 7.6×10^{17} to 2.4×10^{19} training FLOPs. Below roughly 100M parameters MST is *worse* than dense on the compute axes: the coupling and input/output projections are a fixed

overhead that has not yet amortized. We observe no crossover at the upper end and the multiplier is flat to mildly increasing, but we cannot exclude a crossover beyond 914M. This caution is not hypothetical: MoL reports an advantage at 104M that inverts into a 3.01 PPL deficit at 1.3B on the iso-total-parameter axis.

Token ratio. All models are trained at 10.5 tokens per scaling parameter. Partial evidence that the result is not an artifact of this ratio comes from overtrained runs at $2.6\times$ the budget, where the ordering is preserved **[TODO: report these two points; a dense point at matched ratio would make the claim clean]**.

Hardware utilization and decode. MST attains half the utilization of dense, for the two reasons decomposed in Section 5: a measured $2.13\times$ kernel penalty intrinsic to $d_h=32$, and an implementation cost we have not removed. Our wall-clock advantage is therefore smaller than our FLOP advantage. Decode is worse still: it is the regime where per-stream dispatch is least amortized, since there is no sequence dimension over which to hide the N -fold increase in kernel launches, and we measure MST slower than dense there (Table 5). We claim fewer FLOPs per token and faster training; we do not claim faster inference, and a deployment dominated by autoregressive decoding would not benefit from MST as implemented here.

Single seed at scale. Seed variance is characterized at $L=8$ only, where it is $\sigma \leq 0.0003$ BPB over three seeds, two orders of magnitude below every effect we report; the $L \geq 16$ ladder is single-seed per point, as is standard at this scale but worth stating.

Scope. We study pre-training perplexity and zero-shot transfer for decoder-only language models at $T=2048$. We do not study instruction tuning, long-context training, or N scaled jointly with D .

9 CONCLUSION

Narrow parallel blocks are a promising way to restructure the transformer layer, but the way they are usually built, projecting into and out of a full-width residual stream and paying for the projections with sparsity, gives up total-parameter efficiency to buy active-compute efficiency. We showed that if the narrow blocks partition the residual stream instead, the decomposition is free: attention cost and KV cache are preserved exactly, per-layer matmul parameters fall by N , no dispatch machinery is needed, and the resulting dense model dominates a strongly-tuned dense baseline on parameters, inference FLOPs and training compute at once. The remaining design question is not how to route, but how much the columns must talk. Our evidence is that they must talk, and that almost any way of doing so works equally well.

AI USE STATEMENT

Generative AI tools were used in this work as a coding and drafting assistant. Specifically: (i) implementation support for the architecture, training and evaluation code; (ii) analysis scripts that recompute the parameter and FLOP accounting reported in Table 1 and fit the scaling laws in Table 3; (iii) figure generation; and (iv) drafting and editing of the manuscript text. **[TODO: authors: confirm or amend this list, and add any use not covered, e.g. literature search or idea generation]**

Generative AI was not used to generate or alter any experimental result. All numbers reported in this paper come from training runs and from deterministic accounting code executed by the authors; no measurement was produced, imputed or edited by a language model. All AI-assisted code was read, executed and checked against independent references by the authors: the parameter and FLOP accounting was verified against the closed-form derivation of Appendix A, and the scaling fits were reproduced from the raw run table. The authors have reviewed all AI-assisted text and take full responsibility for the final content of this work, including all text, claims and artifacts.

ETHICS STATEMENT

This work studies the pre-training efficiency of decoder-only language models. It involves no human subjects, no personally identifying data and no user study. Training data is a publicly released filtered web corpus (Penedo et al., 2024); we introduce no new dataset and release none. The models

trained here are small by contemporary standards, are not instruction-tuned or deployed, and are not intended or suitable for downstream use; they inherit whatever biases are present in the underlying corpus, and we make no claim that they are safe for application. The contribution is architectural, and its foreseeable effect is to reduce the parameter count and compute required to reach a given language modelling quality. We regard the primary societal consideration as the standard dual-use property of any efficiency result: the same reduction that lowers the cost of research also lowers the cost of producing capable models. We see no concern specific to this architecture beyond that general property. The authors declare no conflicts of interest.

REPRODUCIBILITY STATEMENT

The architecture is specified completely in Section 3: the per-stream sublayers, the coupling of Eq. 2, the geometric window assignment of Section 3.2, and the two optimizer modifications of Section 3.3, with the term-by-term parameter budget in Appendix A. Section 4 fixes the model ladder ($D = 64L$ rounded to a multiple of 128, $N=4$, $T=2048$), the token budget rule (10.5 tokens per scaling parameter, where scaling parameters are transformer matrices plus the output head), and the fact that the dense baseline shares every component and hyper-parameter with MST except the layer structure. Full hyper-parameters and per-run configurations are in Appendix D.

Two points bear directly on reproducing the numbers. First, all cost columns in Table 1 are recomputed for both arms from a single implementation at one commit rather than taken from the original run logs, so the two arms are comparable to each other; the accounting script and the resulting table are released with the code. Second, the ablations of Section 6 are run at $L=8$ with three seeds and we report the seed spread, while the $L \geq 16$ ladder is single-seed per point; Table 6 gives the evidence we have that the $L=8$ ordering transfers. **[TODO: authors: add the code/artifact URL once anonymised, and state the exact commit]**

REFERENCES

- Noah Amsel, David Persson, Christopher Musco, and Robert M Gower. The polar express: Optimal matrix sign methods and their application to the muon algorithm. *arXiv preprint arXiv:2505.16932*, 2025.
- Anonymous. The dual-stream transformer: Channelized architecture for interpretable language modeling. *arXiv preprint arXiv:2603.07461*, 2026a.
- Anonymous. Revisiting the shape convention of transformer language models. *arXiv preprint arXiv:2602.06471*, 2026b.
- Sangmin Bae et al. Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. In *NeurIPS*, 2025.
- Cenk Baykal, Dylan Cutler, Nishanth Dikkala, Nikhil Ghosh, Rina Panigrahy, and Xin Wang. Alternating updates for efficient transformers. In *NeurIPS*, 2023.
- Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- Shane Bergsma et al. Power lines: Scaling laws for weight decay and batch size in llm pre-training. *arXiv preprint arXiv:2505.13738*, 2025.
- Mikhail S Burtsev and Anna Rumshisky. Multi-stream transformers. *arXiv preprint arXiv:2107.10342*, 2021.
- Róbert Csordás, Kazuki Irie, Jürgen Schmidhuber, Christopher Potts, and Christopher D Manning. Moeut: Mixture-of-experts universal transformers. In *NeurIPS*, 2024.
- Damai Dai et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.
- Tri Dao, Beidi Chen, Nimit S Sohoni, Arjun Desai, Michael Poli, Jessica Grogan, Alexander Liu, Aniruddh Rao, Atri Rudra, and Christopher Ré. Monarch: Expressive structured matrices for efficient and accurate training. In *ICML*, 2022.

- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *JMLR*, 23:1–39, 2022.
- Tianyu Fu et al. Moa: Mixture of sparse attention for automatic large language model compression. *arXiv preprint arXiv:2406.14909*, 2024.
- Jordan Hoffmann et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.
- Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. <https://kellerjordan.github.io/posts/muon/>, 2024.
- Andrej Karpathy. nanochat: The best chatgpt that \$100 can buy. <https://github.com/karpathy/nanochat>, 2025.
- Changwoo Lee et al. Blast: Block-level adaptive structured matrices for efficient deep neural network inference. In *NeurIPS*, 2024.
- Deepak Narayanan et al. Efficient large-scale language model training on gpu clusters using megatron-lm. In *SC*, 2021.
- Guilherme Penedo et al. The fineweb datasets: Decanting the web for the finest text data at scale. *NeurIPS Datasets and Benchmarks*, 2024.
- David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. In *ICML*, 2024.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *ICLR*, 2017.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.
- Sainbayar Sukhbaatar, Edouard Grave, Piotr Bojanowski, and Armand Joulin. Adaptive attention span in transformers. *ACL*, 2019.
- Ivan Ternovtsii and Yurii Bilak. Equifinality in mixture of experts: Routing topology does not determine language modeling quality. *arXiv preprint arXiv:2604.14419*, 2026a.
- Ivan Ternovtsii and Yurii Bilak. Mixture of layers with hybrid attention: Parallel thin blocks for sparse transformer compute. *arXiv preprint arXiv:2605.09516*, 2026b.
- Xun Wu, Shaohan Huang, Wenhui Wang, and Furu Wei. Multi-head mixture-of-experts. In *NeurIPS*, 2024.
- Xie et al. Manifold-constrained hyper-connections. *arXiv preprint*, 2026.
- Greg Yang, Edward J Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tensor programs v: Tuning large neural networks via zero-shot hyperparameter transfer. *arXiv preprint arXiv:2203.03466*, 2021.
- Yujiao Yang, Jing Lian, and Linhui Li. Union of experts: Adapting hierarchical routing to equivalently decomposed transformer. *arXiv preprint arXiv:2503.02495*, 2025.
- Minshen Zhang, Xiang Hu, Jianguo Li, Wei Wu, and Kewei Tu. Flash multi-head feed-forward network. *arXiv preprint arXiv:2512.06989*, 2026.
- Yixing Zhang et al. Mswa: Refining local attention with multi-scale window attention. *arXiv preprint arXiv:2501.01039*, 2025.
- Zhanchao Zhou et al. Value residual learning for alleviating attention concentration in transformers. *arXiv preprint arXiv:2410.17897*, 2024.
- Defa Zhu, Hongzhi Huang, Zihao Huang, Yutao Zeng, Yunyao Mao, Banggu Wu, Qiyang Min, and Xun Zhou. Hyper-connections. In *ICLR*, 2025.

A PER-LAYER PARAMETER BUDGET

Term by term, for one layer at width D with N streams of width $d = D/N$. This is the derivation behind Eq. 3.

Table 8: Per-layer parameter budget. The N sub-transformers cost $12D^2/N$ and the coupling costs a further $3D^2/N$, so a layer costs $15D^2/N$ against $12D^2$ dense: a reduction of $0.8N$, not N . The router term Nd is 0.02% of the layer at $D=512$ and is omitted from the totals.

Component	Shapes	Parameters	At $N=4$
Attention, N subs	$4 \times (d \times d)$ each	$4D^2/N$	$1.00 D^2$
FFN, N subs	$(4d \times d) + (d \times 4d)$ each	$8D^2/N$	$2.00 D^2$
Coupling bottleneck	$(D \times d) + (d \times D)$	$2D^2/N$	$0.50 D^2$
Coupling scatter	$N \times (d \times d)$	D^2/N	$0.25 D^2$
Coupling router	$N \times d$	Nd	≈ 0
MST layer		$15D^2/N$	$3.75 D^2$
Dense layer	$4 \times (D \times D) + 8D^2$	$12D^2$	$12.00 D^2$
Reduction		$0.8N$	$3.20 \times$

B COUPLING VARIANTS THAT DID NOT HELP

Every variant below was run on top of the full recipe of Section 3.3 at $L=8$, three seeds each. None improved on the routed bottleneck of Eq. 2. We report them because the pattern is itself the finding: the coupling must exist, but its form is close to irrelevant.

Table 9: Coupling variants that did not improve on the routed bottleneck, $L=8$.

Hypothesis	Variant	Δ BPB	Variant	Δ BPB
Needs nonlinearity	SiLU bottleneck	--	Concat-MLP-split	--
Needs finer routing	Per-slice routing	--	Input-gated router	--
Needs memory	Multi-layer lookahead	--	Hyper-connected residual	--
Needs 2nd order	Bilinear aggregate	--	Cross-sub FFN gating	--
Attention needs cross-sub	Cross-sub KV injection	--	Low-rank query modulation	--
Features stuck in one sub	Cyclic feature permutation	--	Global residual stream	--

C ADDITIONAL ABLATIONS

Table 10: Rows omitted from Table 7: the two optimizer interventions applied separately, and the transition bottleneck width sweep. $L=8$, three seeds.

	Condition	BPB	Δ
Optimizer, separately	Block-diagonal Muon only	--	--
	Per-sub LR only	--	--
Bottleneck width	$\times 1 (d)$	--	--
	$\times 2$	--	--
	$\times 4 (D)$	--	n/a

D EXPERIMENTAL DETAILS

810 **[TODO: full hyper-parameters; per-run configuration hashes; extended ablation table; FLOP**
811 **accounting derivation; seed-variance table]**
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863